Enumerations in C

Months of the Year and Operation Status

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem 1: Months Enumeration

Problem Statement

Create an enumeration for the months of the year and print how many days each month has.

Sample Input and Output

```
Input  : FEB
Output : 28 or 29 days
```

Concept Used

An enumeration (enum) in C is a user-defined data type that holds a fixed set of named integer constants. Here, every month is a constant of the enum Month, and the constants JAN to DEC are numbered 0 to 11 automatically. Because an enum value is just an integer, it is used as an index into two arrays: one holding the month names and one holding the number of days. February is handled as a special case, because it has 28 days in a normal year and 29 days in a leap year.

Algorithm

- Define the enum Month with the constants JAN to DEC, and arrays for the month names and days.
- Read the month name from the user and convert it to upper case using toupper().
- Compare the input with each month name using strcmp() to find the matching enum value.
- If the value is FEB, print "28 or 29 days"; otherwise print the number of days from the array.
- If no month name matches, print an error message.

Source Code (month_days.c)

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Enumeration for the months of the year */
enum Month { JAN, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC };

/* Names and number of days, indexed by the enum values */
const char *monthNames[12] = {"JAN", "FEB", "MAR", "APR", "MAY", "JUN",
                              "JUL", "AUG", "SEP", "OCT", "NOV", "DEC"};
const int monthDays[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

int main(void) {
    char input[20];
    int i, found = 0;

    printf("Enter month (e.g. JAN, FEB): ");
    scanf("%19s", input);

    /* convert input to upper case */
    for (i = 0; input[i] != '\0'; i++) {
        input[i] = toupper((unsigned char)input[i]);
    }

    /* match the input with a month name */
    for (i = JAN; i <= DEC; i++) {
        if (strcmp(input, monthNames[i]) == 0) {
            found = 1;
            if (i == FEB) {
                /* 28 days, or 29 in a leap year */
                printf("Output: 28 or 29 days\n");
            } else {
                printf("Output: %d days\n", monthDays[i]);
            }
            break;
        }
    }

    if (!found) {
        printf("Invalid month entered!\n");
    }
    return 0;
}
```

Output

```
Enter month (e.g. JAN, FEB): FEB
Output: 28 or 29 days
Enter month (e.g. JAN, FEB): JAN
Output: 31 days
Enter month (e.g. JAN, FEB): APR
Output: 30 days
Enter month (e.g. JAN, FEB): xyz
Invalid month entered!
```

2. Problem 2: Operation Status Enumeration

Problem Statement

Define an enumeration with the constants SUCCESS, FAILURE and TIMEOUT, and print a message according to the value given.

Sample Input and Output

```
Input  : FAILURE
Output : Operation Failed
```

Concept Used

The enum Status contains three constants: SUCCESS, FAILURE and TIMEOUT. The text entered by the user is first converted into the matching enum value. A switch statement then works on the enum value, so each constant is matched with its own case and the matching message is printed. This is cleaner than using plain numbers such as 0, 1 and 2, because every value has a meaningful name.

Algorithm

- Define the enum Status with the constants SUCCESS, FAILURE and TIMEOUT.
- Read the status from the user and convert it to upper case.
- Compare the text with each constant name using strcmp() and store the matching enum value.
- Use a switch statement to print "Operation Successful", "Operation Failed" or "Operation Timed Out".
- If the input does not match any constant, print an error message.

Source Code (operation_status.c)

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Enumeration for the result of an operation */
enum Status { SUCCESS, FAILURE, TIMEOUT };

int main(void) {
    char input[20];
    int i;
    enum Status s;

    printf("Enter status (SUCCESS, FAILURE, TIMEOUT): ");
    scanf("%19s", input);

    /* convert input to upper case */
    for (i = 0; input[i] != '\0'; i++) {
        input[i] = toupper((unsigned char)input[i]);
    }

    /* convert the text to an enum value */
    if (strcmp(input, "SUCCESS") == 0) {
        s = SUCCESS;
    } else if (strcmp(input, "FAILURE") == 0) {
        s = FAILURE;
    } else if (strcmp(input, "TIMEOUT") == 0) {
        s = TIMEOUT;
    } else {
        printf("Invalid status entered!\n");
        return 0;
    }

    switch (s) {
        case SUCCESS:
            printf("Output: Operation Successful\n");
            break;
        case FAILURE:
            printf("Output: Operation Failed\n");
            break;
        case TIMEOUT:
            printf("Output: Operation Timed Out\n");
            break;
    }
    return 0;
}
```

Output

```
Enter status (SUCCESS, FAILURE, TIMEOUT): FAILURE
Output: Operation Failed
Enter status (SUCCESS, FAILURE, TIMEOUT): SUCCESS
Output: Operation Successful
Enter status (SUCCESS, FAILURE, TIMEOUT): TIMEOUT
Output: Operation Timed Out
```

3. Conclusion

In this project, enumerations were used to represent fixed groups of related constants. The Month enum showed how enum values can be used as array indexes to look up related data (the number of days), while the Status enum showed how an enum works naturally with a switch statement. Both programs were compiled and tested, and they produced the expected output, including the required cases FEB (28 or 29 days) and FAILURE (Operation Failed).

Using enums makes the code more readable than using plain numbers, and keeps related constants together in one place.